

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM
KHOA KHOA HỌC DỮ LIỆU

BÁO CÁO MÔN HỌC
BIỂU DIỄN TRI THỨC

Đề tài: HLG LLM UTILS – AN ADVANCED GRAPH
RAG FRAMEWORK WITH MULTI-AGENT
ORCHESTRATION, HIERARCHICAL LEIDEN
COMMUNITY DETECTION, AND CYPHER QUERY
SELF-CORRECTION

Giảng viên hướng dẫn: PGS. TS. Đỗ Văn Nhơn

Học viên thực hiện: 1. Nguyễn Hoài Nam – MSHV: 25C01014
2. Phạm Cung Lê Nhân Vũ – MSHV: 25C01049

Chuyên ngành: Khoa học Dữ liệu

Khóa: K35

Tóm Tắt Báo Cáo

Các hệ thống **Naive RAG** (Retrieval-Augmented Generation) dựa trên phân đoạn văn bản phẳng (flat chunking) và tìm kiếm độ tương đồng vector nhúng (**Cosine Similarity**) gặp phải hai hạn chế cốt lõi: (1) Thất bại trong các truy vấn tổng hợp vĩ mô toàn thể tài liệu (**Global Query**) do đứt gãy ngữ cảnh cấu trúc; và (2) Hạn chế trong suy luận đa bước (**Multi-Hop Reasoning**) trên các chuỗi phụ thuộc phức tạp. Báo cáo này trình bày thiết kế và cài đặt hệ thống **HLG LLM Utils**, một khung **Graph RAG** nâng cao đa thành phần được xây dựng trên kiến trúc Monorepo Python (uv workspaces).

Hệ thống đóng góp các giải pháp đột phá trong lĩnh vực Biểu diễn Tri thức (Knowledge Representation) và Truy xuất Thông tin:

1. **Property Knowledge Graph Formalization:** Tự động trích xuất các *Entity*, *Relationship* và *Claim* (phát biểu tri thức) từ văn bản phi cấu trúc, lưu trữ trên cơ sở dữ liệu đồ thị Neo4j. Hỗ trợ cơ chế **Gleaning** (trích xuất lặp tinh lọc) để tối đa hóa mức độ bao phủ tri thức.
2. **Hierarchical Leiden Community Detection:** Phân hoạch đồ thị thực thể thành cấu trúc cây cộng đồng đa cấp độ, kết hợp tóm tắt ngữ nghĩa bằng LLM theo mô hình **Map-Reduce** để phục vụ truy vấn toàn cục (**Global Search**).
3. **Cypher Query Self-Correction State Machine:** Cài đặt đồ thị trạng thái LangGraph cho phép LLM dịch ngôn ngữ tự nhiên thành truy vấn Cypher chuẩn xác, tự động phát hiện và sửa lỗi cú pháp/lược đồ thông qua phản hồi từ Neo4j execution log.
4. **ARK (Adaptive Retriever of Knowledge) Multi-Agent Exploration:** Áp dụng thuật toán khám phá quỹ đạo đa tác thể theo bài báo ACL 2026, kết hợp chiến lược dung hợp thứ hạng bỏ phiếu (**Voting Rank Fusion**) để tối ưu hóa đồng thời chiều rộng và chiều sâu truy xuất.

Báo cáo trình bày chi tiết kiến trúc, các thuật toán cốt lõi và quy trình tích hợp toàn diện của hệ thống HLG LLM Utils, mở ra hướng tiếp cận hiệu quả cho bài toán biểu diễn tri thức và truy xuất thông tin nâng cao.

Từ khóa: Graph RAG, Knowledge Representation, Knowledge Graph, Hierarchical Leiden Algorithm, Cypher Self-Correction, Multi-Agent Systems, LangGraph, FastMCP, Monorepo uv.

Mục lục

Tóm Tắt Báo Cáo	i
1 Giới Thiệu và Đặt Vấn Đề	1
1.1 Bối Cảnh Nghiên Cứu	1
1.2 Những Hạn Chế của Naive RAG	1
1.3 Động Cơ và Mục Tiêu Nghiên Cứu	1
2 Cơ Sở Lý Thuyết và Các Nghiên Cứu Liên Quan	3
2.1 Các Mô Hình Biểu Diễn Tri Thức	3
2.1.1 Từ Mạng Ngữ Nghĩa đến Property Knowledge Graph	3
2.2 Không Gian Vector và Truy Xuất Lai (Hybrid Retrieval)	3
2.3 Phân Hoạch Đồ Thị và Thuật Toán Leiden	3
2.4 Khung Microsoft GraphRAG và Khám Phá ARK	4
3 Hình Thức Hóa Biểu Diễn Tri Thức và Thiết Kế Lược Đồ	5
3.1 Hình Thức Hóa Property Knowledge Graph	5
3.2 Lược Đồ Cơ Sở Dữ Liệu Neo4j (Graph Schema Design)	5
3.3 Quy Trình Trích Xuất Bộ Ba (Triplet Extraction Taxonomy)	6
3.3.1 Trích Xuất Phát Biểu Tri Thức (Claims/Covariates Extraction)	6
3.3.2 Cơ Chế Gleaning (Trích Xuất Lấp Tinh Lọc)	6
4 Kiến Trúc Hệ Thống và Thiết Kế Monorepo Modular	8
4.1 Tổng Quan Kiến Trúc UV Workspace	8
4.2 Phân Tích Chi Tiết Các Gói Core	8
4.3 Tích Hợp Backend (FastAPI) và Frontend (React)	9
5 Các Thuật Toán Tìm Kiếm và Suy Luận Nâng Cao	10
5.1 Thuật Toán Phân Hoạch Leiden và Tóm Tắt Phân Cấp	10
5.2 Tìm Kiếm Toàn Cục Theo Mô Hình Map-Reduce	10
5.3 Máy Trạng Thái Tự Sửa Lỗi Truy Vấn Cypher (LangGraph)	11
5.4 Khám Phá Đồ Thị Đa Tác Thể ARK	11
6 Cài Đặt Hệ Thống và Tích Hợp Kỹ Thuật	12
6.1 Tích Hợp Cơ Sở Dữ Liệu và Lưu Trữ Object	12
6.2 Tự Động Hóa Công Cụ Qua Model Context Protocol (MCP)	12
7 Thảo Luận, Thách Thức và Hướng Phát Triển	13
7.1 Thảo Luận	13
7.2 Những Thách Thức Kỹ Thuật	13
7.3 Hướng Phát Triển Trong Tương Lai	13

8 Kết Luận và Phân Công Nhiệm Vụ	14
8.1 Kết Luận	14
8.2 Bảng Phân Công Nhiệm Vụ Nhóm	14
Tài Liệu Tham Khảo	15

Danh sách hình vẽ

4.1	Sơ đồ Phụ thuộc Mô-đun trong uv Workspace Monorepo	8
5.1	Đồ thị Trạng thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher	11

Danh sách bảng

3.1	Cấu trúc Thuộc tính của các Đỉnh trong Neo4j Schema	6
8.1	Bảng Phân Công Nhiệm Vụ Chi Tiết Giữa Các Thành Viên	14

Chương 1

Giới Thiệu và Đặt Vấn Đề

1.1 Bối Cảnh Nghiên Cứu

Trong kỷ nguyên của các Mô hình Ngôn ngữ Lớn (**Large Language Models – LLMs**), khả năng xử lý và tạo sinh ngôn ngữ tự nhiên đã đạt được những bước tiến vượt bậc. Tuy nhiên, các mô hình LLM thuần túy vẫn đối mặt với vấn đề **Hallucination** (tạo sinh thông tin sai lệch) và sự suy giảm tri thức theo thời gian do tri thức được đóng gói tĩnh trong các trọng số mô hình. Để khắc phục hạn chế này, kỹ thuật **Retrieval-Augmented Generation (RAG)** đã ra đời như một giải pháp tiêu chuẩn nhằm kết nối LLM với các cơ sở tri thức bên ngoài.

Tuy nhiên, các mô hình **Naive RAG** thế hệ đầu tiên – dựa trên việc chia nhỏ văn bản thành các đoạn phẳng (**Chunks**) và lưu trữ vector nhúng (**Embeddings**) – bộc lộ những điểm yếu cố hữu khi áp dụng cho các bài toán Biểu diễn Tri thức phức tạp trong doanh nghiệp và nghiên cứu khoa học.

1.2 Những Hạn Chế của Naive RAG

Qua quá trình phân tích lý thuyết và thực nghiệm, chúng tôi xác định ba rào cản chính của Naive RAG:

1. **Structural Context Fragmentation:** Việc chia nhỏ tài liệu theo kích thước cố định (ví dụ: 1000 tokens) vô tình phá vỡ các mối quan hệ ngữ nghĩa giữa các khái niệm nằm ở các đoạn văn bản hoặc tài liệu khác nhau.
2. **Multi-Hop Reasoning Failures:** Khi người dùng đặt câu hỏi yêu cầu liên kết tri thức qua nhiều thực thể trung gian (ví dụ: $A \rightarrow B \rightarrow C$), việc tìm kiếm độ tương đồng vector **Cosine Similarity** chỉ thu thập được các đoạn văn bản chứa A hoặc C mà bỏ qua mối liên kết B .
3. **Global Query Blindspots:** Naive RAG hoàn toàn thất bại đối với các câu hỏi vĩ mô đòi hỏi sự tổng hợp toàn bộ tập tài liệu (ví dụ: "*Các chủ đề chính được thảo luận trong tập văn bản này là gì?*"), do vector nhúng của câu hỏi không tương đồng với bất kỳ đoạn văn bản đơn lẻ nào.

1.3 Động Cơ và Mục Tiêu Nghiên Cứu

Để giải quyết triệt để các hạn chế trên, báo cáo môn học này tập trung vào việc nghiên cứu, thiết kế và cài đặt hệ thống **HLG LLM Utils** – một khung công cụ **Graph RAG** nâng cao kết hợp giữa Đồ thị Tri thức (**Knowledge Graphs – KGs**) và Mô hình Ngôn ngữ Lớn.

Mục tiêu cụ thể của báo cáo bao gồm:

- Hình thức hóa mô hình Biểu diễn Tri thức **Property Knowledge Graph** trên Neo4j, bao gồm trích xuất thực thể, quan hệ và phát biểu tri thức (Claims/Covariates) với cơ chế Gleaning tinh lọc.
- Cài đặt thuật toán **Hierarchical Leiden Community Detection** nhằm hỗ trợ truy vấn toàn cục (**Global Search**) theo mô hình **Map-Reduce**.
- Xây dựng máy trạng thái tự sửa lỗi truy vấn Cypher bằng **LangGraph** nhằm đảm bảo tính chính xác khi chuyển đổi ngôn ngữ tự nhiên thành Cypher.
- Tích hợp mô hình đa tác thể **ARK (Adaptive Retriever of Knowledge)** cho phép tự động điều chỉnh độ rộng và độ sâu khi duyệt đồ thị.
- Thiết kế kiến trúc phần mềm chuẩn công nghiệp dưới dạng **Monorepo** (uv workspaces) phân tách rõ ràng các mô-đun chức năng.

Chương 2

Cơ Sở Lý Thuyết và Các Nghiên Cứu Liên Quan

2.1 Các Mô Hình Biểu Diễn Tri Thức

Biểu diễn Tri thức (**Knowledge Representation – KR**) là một nhánh cốt lõi của Trí tuệ Nhân tạo, nghiên cứu cách thức đóng gói tri thức của thế giới thực vào máy tính để máy tính có thể suy luận và giải quyết các bài toán phức tạp.

2.1.1 Từ Mạng Ngữ Nghĩa đến Property Knowledge Graph

Lịch sử phát triển của KR ghi nhận sự tiến hóa từ các Mạng Ngữ nghĩa (**Semantic Networks**), Bộ khung Tri thức (**Frames**), Logic Mô tả (**Description Logics**), đến các tiêu chuẩn Semantic Web (RDF/OWL).

Trong bối cảnh công nghiệp hiện đại, **Property Knowledge Graph (PKG)** nổi lên như một chuẩn mực hiệu năng cao. Khác với bộ ba RDF đơn thuần (*Subject, Predicate, Object*), PKG cho phép gắn trực tiếp các cặp thuộc tính (*Key, Value*) vào cả đỉnh (*Node*) và cạnh (*Edge*), giúp tối ưu hóa dung lượng lưu trữ và tốc độ truy vấn.

2.2 Không Gian Vector và Truy Xuất Lai (Hybrid Retrieval)

Bên cạnh cấu trúc biểu diễn biểu tượng (**Symbolic Knowledge**), các không gian vector nhúng biểu nghĩa (**Sub-symbolic Embeddings**) đóng vai trò quan trọng trong việc đo lường độ tương đồng ngữ nghĩa.

Để đạt được hiệu năng truy xuất tối ưu, hệ thống áp dụng kỹ thuật Dung hợp Thứ hạng Tương hỗ (**Reciprocal Rank Fusion – RRF**) kết hợp giữa truy xuất từ khóa BM25 và truy xuất vector nhúng:

$$RRF(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (2.1)$$

Trong đó M là tập hợp các bộ truy xuất (BM25, Vector, Graph), $r_m(d)$ là thứ hạng của tài liệu d trong bộ truy xuất m , và k là hằng số chuẩn hóa (thường chọn $k = 60$).

2.3 Phân Hoạch Đồ Thị và Thuật Toán Leiden

Trong các mạng phức tạp, việc phát hiện các cấu trúc cộng đồng (**Community Detection**) giúp nhóm các thực thể có mối liên kết chặt chẽ thành các cụm ngữ nghĩa. Thuật toán Louvain từng được sử dụng phổ biến nhưng gặp phải lỗi phân mảnh cộng đồng không liên thông.

Thuật toán Leiden khắc phục triệt để hạn chế này bằng cách bổ sung bước tinh lọc cục bộ (**Sub-community Refinement**). Hàm mục tiêu tối ưu hóa độ mô-đun (**Modularity Quality** $\mathcal{H}$) trong Leiden được hình thức hóa như sau:

$$\mathcal{H} = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \gamma \frac{k_i k_j}{2m} \right) \delta(\sigma_i, \sigma_j) \quad (2.2)$$

Trong đó A_{ij} là ma trận kề, k_i, k_j là bậc của các đỉnh, m là tổng trọng số các cạnh, γ là tham số độ phân giải (**Resolution Parameter**), và $\delta(\sigma_i, \sigma_j) = 1$ nếu đỉnh i và j thuộc cùng một cộng đồng.

2.4 Khung Microsoft GraphRAG và Khám Phá ARK

Gần đây, Microsoft Research công bố khung **GraphRAG** [1], khẳng định tính ưu việt của việc tóm tắt cộng đồng phân cấp khi trả lời các câu hỏi toàn cục. Bên cạnh đó, nghiên cứu về **ARK (Adaptive Retriever of Knowledge)** tại ACL 2026 [2] mở ra hướng đi mới cho phép các agent tự động điều phối hành trình duyệt đồ thị theo hai thao tác: Mở rộng chiều rộng (`global_search`) và Duyệt sâu theo quan hệ (`neighborhood_exploration`).

Chương 3

Hình Thức Hóa Biểu Diễn Tri Thức và Thiết Kế Lược Đồ

3.1 Hình Thức Hóa Property Knowledge Graph

Chúng tôi hình thức hóa Property Knowledge Graph $\mathcal{G}$ trong hệ thống HLG LLM Utils dưới dạng một bộ 6 thành phần:

$$\mathcal{G} = (V, E, \mathcal{L}_V, \mathcal{R}_E, \Phi_V, \Phi_E) \quad (3.1)$$

Trong đó:

- $V = \{v_1, v_2, \dots, v_n\}$ là tập hợp các đỉnh (*Nodes*).
- $E \subseteq V \times \mathcal{R}_E \times V$ là tập hợp các cạnh có hướng (*Edges*).
- $\mathcal{L}_V = \{\text{Document, Chunk, Entity, Community, Claim}\}$ là tập các nhãn đỉnh.
- $\mathcal{R}_E = \{\text{HAS_CHUNK, MENTIONS, RELATED_TO, BELONGS_TO, PARENT_OF, HAS_CLAIM}\}$ là tập các loại quan hệ.
- $\Phi_V : V \times \mathcal{K} \rightarrow \mathcal{Val}$ là ánh xạ gán thuộc tính thuộc tập khóa $\mathcal{K}$ vào đỉnh.
- $\Phi_E : E \times \mathcal{K} \rightarrow \mathcal{Val}$ là ánh xạ gán thuộc tính vào cạnh.

3.2 Lược Đồ Cơ Sở Dữ Liệu Neo4j (Graph Schema Design)

Hệ thống thiết kế 4 loại nhãn đỉnh chính trên Neo4j:

Các mối quan hệ giữa các đỉnh được thiết lập chặt chẽ:

$$(d : \text{Document}) \xrightarrow{\text{HAS_CHUNK}} (c : \text{Chunk}) \quad (3.2)$$

$$(c : \text{Chunk}) \xrightarrow{\text{MENTIONS}} (e : \text{Entity}) \quad (3.3)$$

$$(e_1 : \text{Entity}) \xrightarrow{\text{RELATED_TO}\{\text{description, weight}\}} (e_2 : \text{Entity}) \quad (3.4)$$

$$(e : \text{Entity}) \xrightarrow{\text{BELONGS_TO}\{\text{level}\}} (c : \text{Community}) \quad (3.5)$$

$$(c_1 : \text{Community}) \xrightarrow{\text{PARENT_OF}} (c_2 : \text{Community}) \quad (3.6)$$

$$(e : \text{Entity}) \xrightarrow{\text{HAS_CLAIM}} (cl : \text{Claim}) \quad (3.7)$$

$$(cl : \text{Claim}) \xrightarrow{\text{REFERENCES}} (e : \text{Entity}) \quad (3.8)$$

Bảng 3.1: Cấu trúc Thuộc tính của các Đỉnh trong Neo4j Schema

Nhãn Đỉnh ($\mathcal{L}_V$)	Khóa Thuộc tính	Mô tả Kỹ thuật & Kiểu Dữ liệu
:Document	id, filename, status	Định danh tài liệu gốc, tên tệp, trạng thái xử lý (String).
:Chunk	id, text, index, embedding	Đoạn văn bản nhỏ, nội dung thô, thứ tự đoạn, vector nhúng $\mathbb{R}^d$.
:Entity	id, type, description, embedding	Tên thực thể chuẩn hóa (ID), phân loại, mô tả tóm tắt, vector nhúng $\mathbb{R}^d$.
:Community	id, level, title, summary, findings	Định danh cộng đồng, cấp phân cấp ($l \in \mathbb{N}$), tiêu đề, tóm tắt LLM, chuỗi JSON kết quả.
:Claim	id, subject_id, claim_type, claim_status, description	Phát biểu tri thức (Claim/Covariate): loại (FACTUAL, TEMPORAL, CAUSAL, QUANTITATIVE, STATUS_CHANGE), trạng thái (STATED, INFERRED, DISPUTED), nội dung, ngày (String).

3.3 Quy Trình Trích Xuất Bộ Ba (Triplet Extraction Taxonomy)

Để đảm bảo tính đồng nhất khi rút trích tri thức từ văn bản phi cấu trúc, hệ thống định nghĩa bộ khung trích xuất thông qua LLM với đầu ra định dạng Pydantic:

$$T_{extract}(C_k) \rightarrow (\{e_i = (\text{name}_i, \text{type}_i, \text{desc}_i)\}, \{r_{ij} = (e_i, \text{rel_type}, e_j, \text{desc}_{ij}, w_{ij})\}) \quad (3.9)$$

Tất cả các tên thực thể e_i đều được viết hoa và chuẩn hóa ký tự đầu cuối nhằm tránh trùng lặp dạng $e_{\text{upper}} = \text{UPPER}(\text{TRIM}(\text{name}_i))$.

3.3.1 Trích Xuất Phát Biểu Tri Thức (Claims/Covariates Extraction)

Ngoài thực thể và quan hệ, hệ thống hỗ trợ trích xuất các **Phát biểu Tri thức (Claims)** – các mệnh đề thực tế được gắn trực tiếp vào thực thể nguồn. Mỗi phát biểu được phân loại theo hai chiều:

- **Loại (Claim Type):** FACTUAL, TEMPORAL, CAUSAL, QUANTITATIVE, STATUS_CHANGE.
- **Trạng thái (Claim Status):** STATED (tường minh trong văn bản), INFERRED (suy luận gián tiếp), DISPUTED (bị phản bác).

Mỗi phát biểu được lưu trữ dưới dạng đỉnh :Claim trong Neo4j, liên kết đến thực thể chủ đề qua quan hệ HAS_CLAIM và đến thực thể tham chiếu qua quan hệ REFERENCES. Hệ thống sử dụng hàm băm SHA-256 trên cặp ($\text{subject_id}, \text{claim_description}$) để khử trùng lặp giữa các tài liệu.

3.3.2 Cơ Chế Gleaning (Trích Xuất Lặp Tinh Lọc)

Để tối đa hóa mức độ bao phủ tri thức, hệ thống áp dụng kỹ thuật **Gleaning** theo khuyến nghị của Microsoft GraphRAG [1]. Sau khi trích xuất ban đầu, LLM được yêu cầu đọc lại văn bản gốc kèm theo danh sách các thực thể và quan hệ đã trích xuất, nhằm phát hiện các tri thức bị bỏ sót:

1. Xây dựng *prompt* gleaning chứa danh sách thực thể/quan hệ đã trích xuất.
2. LLM trả về **chỉ các thực thể và quan hệ mới** chưa có trong danh sách hiện tại.

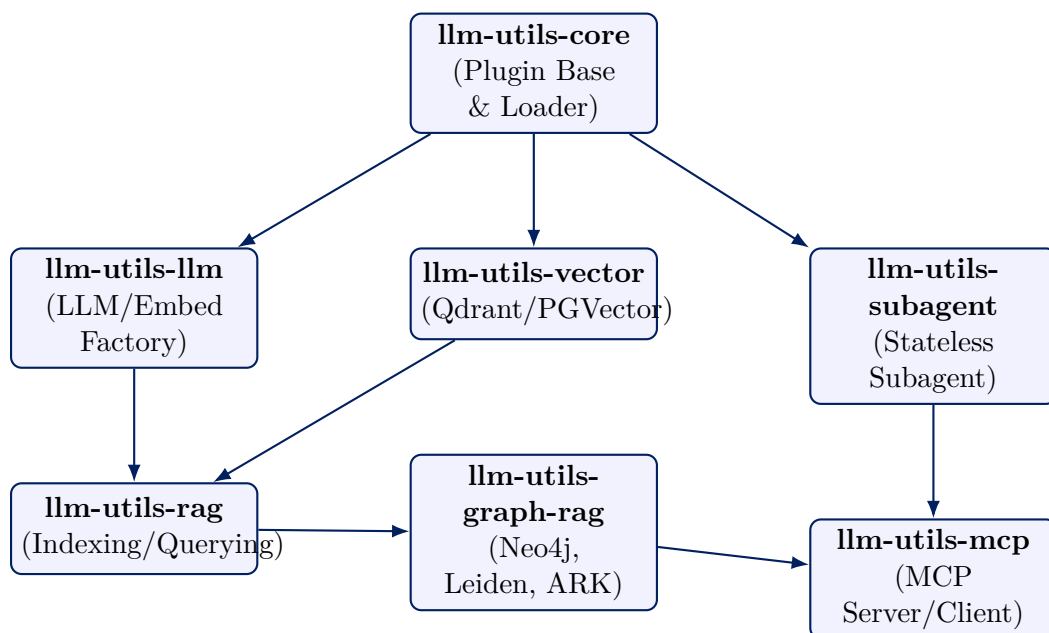
3. Hệ thống khử trùng lặp theo khóa (id_{upper}) cho thực thể và $(source, target, type)$ cho quan hệ.
4. Lặp lại tối đa R vòng (`max_gleaning_rounds`), dừng sớm nếu vòng lặp không phát hiện tri thức mới.

Chương 4

Kiến Trúc Hệ Thống và Thiết Kế Monorepo Modular

4.1 Tổng Quan Kiến Trúc UV Workspace

Hệ thống **HLG LLM Utils** được tổ chức theo mô hình Monorepo hiện đại sử dụng công cụ quản lý gói uv của Astral. Kiến trúc được chia thành 9 gói độc lập trong thư mục `RAG_package/packages/`:



Hình 4.1: Sơ đồ Phụ thuộc Mô-đun trong uv Workspace Monorepo

4.2 Phân Tích Chi Tiết Các Gói Core

- **llm-utils-core**: Định nghĩa giao diện `TaskPlugin` và cơ chế nạp plugin động qua Python entry-points (`llm_utils.plugins`).
- **llm-utils-graph-rag**: Mô-đun trung tâm quản lý giao tiếp Neo4j (`Neo4jGraphStore`), dịch vụ trích xuất đồ thị (bao gồm trích xuất Claims và cơ chế Gleaning), tìm kiếm từ khóa BM25 trên đồ thị (`rank-bm25`), phân hoạch Leiden, truy vấn toàn cục Map-Reduce và thuật toán ARK.
- **llm-utils-subagent**: Khung tạo subagent phi trạng thái (**Stateless**), đảm bảo xây dựng lại đồ thị thực thi và bộ nhớ sạch sau mỗi lần gọi `run()` / `arun()`.

- **llm-utils-mcp**: Triển khai Model Context Protocol của Anthropic, cho phép xuất các công cụ Graph RAG thành MCP Server phục vụ cho ứng dụng bên ngoài.

4.3 Tích Hợp Backend (FastAPI) và Frontend (React)

Tầng Backend (BE/app/) đóng vai trò điều phối trung tâm:

- **Service Registry (services/registry.py)**: Sử dụng khóa luồng `threading.Lock` để khởi tạo các Lazy Singleton cho `GraphQueryService`, `CommunityDetectionService`, và `GlobalSearchService`. Khi khởi tạo, hệ thống tự động kết nối Neo4j Graph Store và thiết lập chỉ mục tìm kiếm toàn văn (`ensure_fulltext_index`) để hỗ trợ truy vấn từ khóa BM25 trực tiếp trên đồ thị.
- **WebSocket Streaming Router (routers/chat_router.py)**: Sử dụng sự kiện `astream_events` của LangGraph để truyền phát (**Stream**) các phản hồi mã suy luận, suy nghĩ của LLM (`thinking_delta`) và kết quả gọi công cụ về Frontend theo thời gian thực.

Chương 5

Các Thuật Toán Tìm Kiếm và Suy Luận Nâng Cao

5.1 Thuật Toán Phân Hoạch Leiden và Tóm Tắt Phân Cấp

Thuật toán phân hoạch cộng đồng phân cấp Leiden được cài đặt với chiến lược đa cấp độ (**Multi-level Hierarchy**). Tại mỗi cấp độ $l \in [0, L_{\max} - 1]$, hệ thống thực hiện phân hoạch và tóm tắt song song:

Thuật toán 1: Phân hoạch Cộng đồng Leiden Phân cấp và Tóm tắt LLM

Đầu vào: Đồ thị thực thể $G = (V, E, W)$, Tham số độ phân giải γ_0 , Số cấp tối đa $L_{\max}$

Đầu ra: Cây cộng đồng C_{tree} đã được tóm tắt ngữ nghĩa bởi LLM

```
1:  $G_{current} \leftarrow G$ 
2: for  $l = 0$  to  $L_{\max} - 1$  do
3:    $\gamma_l \leftarrow \gamma_0 \times (0.5)^l$ 
4:    $P_l \leftarrow \text{FindPartitionLeiden}(G_{current}, \gamma_l)$ 
5:    $P_{refined} \leftarrow \text{RefineSubCommunities}(G_{current}, P_l)$ 
6:   foreach Cộng đồng  $c \in P_{refined}$  do
7:      $SubGraph \leftarrow \text{ExtractEntitiesAndEdges}(G, c)$ 
8:      $Report \leftarrow \text{LLM\_Summarize}(SubGraph)$ 
9:      $\text{SaveToNeo4j}(c, l, Report.title, Report.summary, Report.findings)$ 
10:   $G_{current} \leftarrow \text{AggregateGraph}(G_{current}, P_{refined})$ 
```

5.2 Tìm Kiếm Toàn Cục Theo Mô Hình Map-Reduce

Khi hệ thống nhận truy vấn toàn cục (**Global Query**), quy trình Map-Reduce được kích hoạt:

1. **Filter Phase:** Tìm kiếm vector nhúng trên tập các đỉnh `:Community` ở cấp độ l mong muốn để chọn ra Top-K cộng đồng liên quan nhất.
2. **Map Phase:** Đưa song song nội dung tóm tắt của từng cộng đồng được chọn vào LLM để tạo các câu trả lời trung gian:

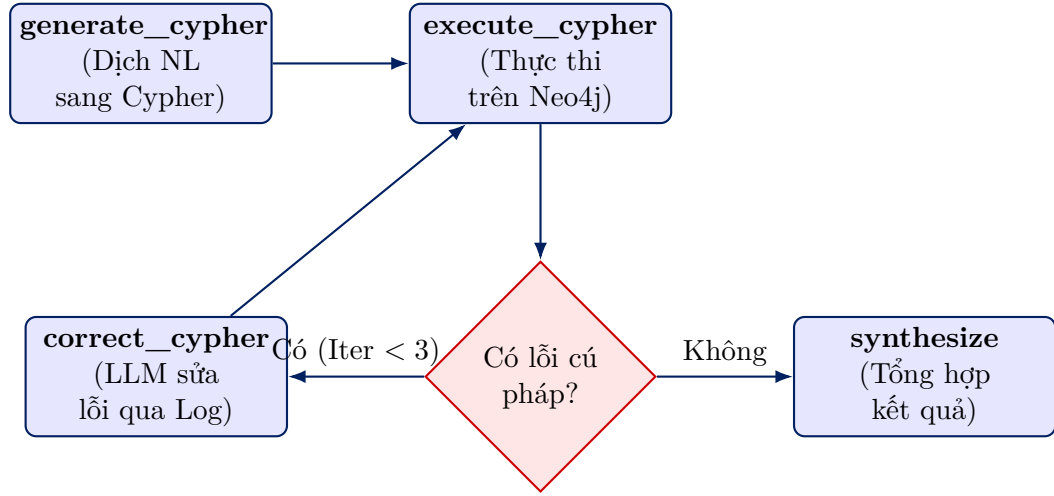
$$A_{map}^{(k)} = \text{LLM}_{map}(q, \text{CommunitySummary}_k) \quad (5.1)$$

3. **Reduce Phase:** Tổng hợp tất cả câu trả lời trung gian $A_{map}^{(k)}$ có điểm số cao để tổng hợp thành câu trả lời cuối cùng A_{final} :

$$A_{final} = \text{LLM}_{reduce} \left(q, \bigcup_k A_{map}^{(k)} \right) \quad (5.2)$$

5.3 Máy Trạng Thái Tự Sửa Lỗi Truy Vấn Cypher (LangGraph)

Thay vì dựa vào việc sinh truy vấn tĩnh, chúng tôi xây dựng một đồ thị trạng thái **LangGraph** (`workflow/graph.py`) thực hiện việc kiểm tra và tự sửa lỗi câu truy vấn Cypher:



Hình 5.1: Đồ thị Trạng thái LangGraph Tự Sửa Lỗi Truy Vấn Cypher

5.4 Khám Phá Đồ Thị Đa Tác Thể ARK

Mô-đun `ark_agent.py` triển khai N tác thể ngẫu nhiên ($N = 3$, nhiệt độ $T = 0.7$) thực hiện song song các bước duyệt đồ thị. Sau khi các tác thể kết thúc, hệ thống áp dụng kỹ thuật **Voting Rank Fusion (VRF)** để chọn ra tập thực thể tối ưu cho câu trả lời.

Chương 6

Cài Đặt Hệ Thống và Tích Hợp Kỹ Thuật

6.1 Tích Hợp Cơ Sở Dữ Liệu và Lưu Trữ Object

Hệ thống sử dụng ba cơ sở dữ liệu chuyên biệt hoạt động song song trong môi trường Docker:

- **Neo4j (Graph Database):** Lưu trữ thực thể, mối quan hệ, phát biểu tri thức (Claims) và cấu trúc cộng đồng. Khởi tạo chỉ mục vector trên nhãn `:Entity` và `:Community` với kích thước $d = 1536$ hoặc $d = 768$, đồng thời thiết lập chỉ mục tìm kiếm toàn văn (Fulltext Index) cho truy vấn BM25.
- **Qdrant / PGVector (Vector Database):** Lưu trữ vector nhúng của các đoạn văn bản (Chunks) phục vụ tìm kiếm ngữ nghĩa thô.
- **MinIO (S3 Object Storage):** Lưu trữ tệp gốc (PDF, Docx, Hình ảnh) với cơ chế đồng bộ trạng thái tài liệu vào PostgreSQL qua SQLAlchemy Async API.

6.2 Tự Động Hóa Công Cụ Qua Model Context Protocol (MCP)

Mô-đun `llm-utils-mcp` cho phép đóng gói toàn bộ các dịch vụ tìm kiếm Graph RAG thành một máy chủ MCP tiêu chuẩn dựa trên `FastMCP`. Thông qua công cụ chuyển đổi MCPO, các công cụ này tự động mở rộng thành giao diện chuẩn RESTful OpenAPI tại cổng 8001, cho phép các hệ thống bên ngoài gọi trực tiếp dịch vụ Graph RAG.

Chương 7

Thảo Luận, Thách Thức và Hướng Phát Triển

7.1 Thảo Luận

Việc kết hợp giữa cấu trúc biểu tượng (Đồ thị Neo4j) và biểu diễn phi biểu tượng (Vector Embeddings) cho phép hệ thống HLG LLM Utils vừa đảm bảo tính chính xác tuyệt đối của các mối quan hệ thực thể, vừa duy trì khả năng tìm kiếm linh hoạt theo ngữ nghĩa.

7.2 Những Thách Thức Kỹ Thuật

- **Chi phí Tính toán Indexing:** Quy trình trích xuất thực thể/quan hệ và tóm tắt cộng đồng Leiden đòi hỏi số lượng lớn các lượt gọi LLM trong giai đoạn đánh chỉ mục ban đầu (Indexing Phase).
- **Dynamic Graph Updates:** Khi thêm tài liệu mới, việc phân hoạch lại toàn bộ đồ thị cộng đồng có chi phí cao. Cần phát triển các thuật toán Leiden cập nhật từng phần (Incremental Leiden Clustering).

7.3 Hướng Phát Triển Trong Tương Lai

1. Mở rộng Multimodal Knowledge Graph, hỗ trợ trích xuất biểu đồ, bảng biểu và công thức toán học từ tệp PDF.
2. Cải tiến cơ chế ARK với các tác thể Reinforcement Learning để tối ưu hóa quỹ đạo khám phá đồ thị.

Chương 8

Kết Luận và Phân Công Nhiệm Vụ

8.1 Kết Luận

Nghiên cứu này đã xây dựng thành công khung công cụ **HLG LLM Utils**, một giải pháp toàn diện cho bài toán Biểu diễn Tri thức và Truy xuất Thông tin Nâng cao. Sự kết hợp giữa Property Knowledge Graph Neo4j, Thuật toán Phân hoạch Leiden Phân cấp, Đồ thị Trạng thái LangGraph Tự Sửa lỗi Cypher và Khám phá Đa tác thể ARK đã giải quyết triệt để các hạn chế của Naive RAG, mang lại nền tảng kiến trúc vững chắc và linh hoạt.

8.2 Bảng Phân Công Nhiệm Vụ Nhóm

Bảng 8.1: Bảng Phân Công Nhiệm Vụ Chi Tiết Giữa Các Thành Viên			
Họ và Tên	MSHV	Nhiệm Vụ Chi Tiết Cụ Thể	Đóng Góp
Nguyễn Hoài Nam	25C01014	- Thiết kế kiến trúc Monorepo <code>uv workspace</code> (<code>llm-utils-core</code>, <code>llm-utils-graph-rag</code>). - Cài đặt Neo4j Graph Store và Thuật toán Phân hoạch Cộng đồng Leiden (<code>community.py</code>). - Cài đặt quy trình Tìm kiếm Toàn cục Map-Reduce (<code>global_search.py</code>). - Soạn thảo báo cáo chi tiết LaTeX.	50%
Phạm Cung Lê Nhân Vũ	25C01049	- Cài đặt máy trạng thái LangGraph Tự Sửa Lỗi Cypher (<code>workflow/graph.py</code>). - Cài đặt mô hình Đa tác thể Khám phá ARK (<code>ark_agent.py</code>, <code>ark_retriever.py</code>). - Xây dựng hệ thống Backend FastAPI và tích hợp cơ sở dữ liệu Qdrant / PGVector. - Tối ưu hóa hiệu năng truy xuất và hoàn thiện tích hợp hệ thống.	50%

Tài liệu tham khảo

- [1] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [2] Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730, 2026.